

AWS - Serverless Architecture & Cloud Automation

Test Name - Serverless architecture

Created by - Saleem Shaikh

Batch - 16A

Submitted Date - May 31 2026

Skill Test Github Link -

I have picked up the following assignments:

- 1) Assignment 1: Automated Instance Management Using AWS Lambda and Boto3 - **Complete**
- 2) Assignment 2: Automated S3 Bucket Cleanup Using AWS Lambda and Boto3
- 3) Assignment 6: Monitor and Alert High AWS Billing Using AWS Lambda, Boto3, and SNS
- 4) Assignment 12: Auto-Scale EC2 Instances Based on Load Using AWS Lambda, Boto3, and SNS
- 5) Assignment 19: Load Balancer Health Checker

1) Assignment 1: Automated Instance Management Using AWS Lambda and Boto3

Objective: In this assignment, you will gain hands-on experience with AWS Lambda and Boto3, Amazon's SDK for Python. You will create a Lambda function that will automatically manage EC2 instances based on their tags.

Task: You're tasked to automate the stopping and starting of EC2 instances based on tags. Specifically:

1. EC2 Setup:

- Navigate to the EC2 dashboard and create two new t2.micro instances (or any other available free-tier type).
- Tag the first instance with a key `Action` and value `Auto-Stop`.
- Tag the second instance with a key `Action` and value `Auto-Start`.

```
testing@inblrlt-002:~/study/assignment$ aws ec2 describe-instances --query "Reservations[*].Instances[*].{PublicAddress:
anceId,Platform:PlatformDetails,State:State.Name,Name:Tags[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value
"
DescribeInstances
+-----+-----+-----+-----+-----+-----+-----+
| Action | InstanceID | Name | Platform | PrivateAddress | PublicAddress | State |
+-----+-----+-----+-----+-----+-----+-----+
| None | i-037f542b93fe8accd | frontend-01 | Linux/UNIX | 172.31.12.201 | None | stopped |
| None | i-0400124bba6643b1a | frontend-02 | Linux/UNIX | 172.31.13.171 | None | stopped |
| Auto-Stop | i-011facb3f85dcfcde | dev-server | Linux/UNIX | 172.31.8.179 | 43.204.230.177 | running |
| Auto-Start | i-03cea284f141c9cba | prod-server | Linux/UNIX | 172.31.15.137 | None | stopped |
+-----+-----+-----+-----+-----+-----+-----+
testing@inblrlt-002:~/study/assignment$
```

2. Lambda IAM Role:

- In the IAM dashboard, create a new role for Lambda.
- Attach the `AmazonEC2FullAccess` policy to this role. (Note: In a real-world scenario, you would want to limit permissions for better security.)
Created an inline policy with limited permissions (describe/stop/start).

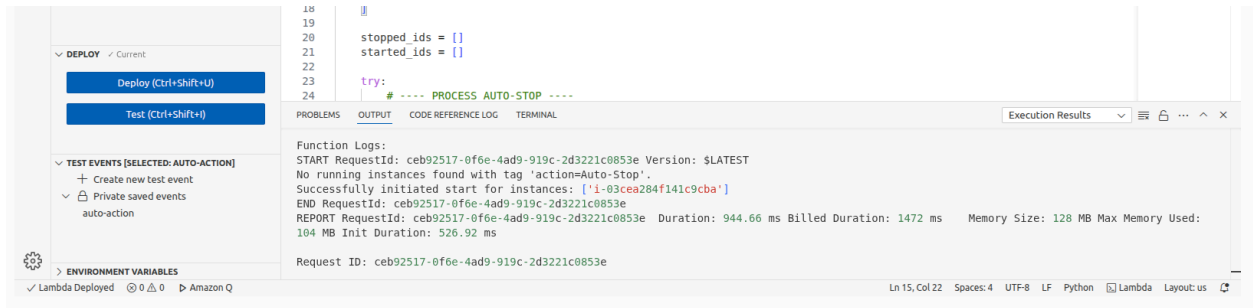
```

assignment > {} start-stop-iam-policy > [ ] Statement
1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Effect": "Allow",
6              "Action": [
7                  "ec2:StartInstances",
8                  "ec2:StopInstances"
9              ],
10             "Resource": "arn:aws:ec2:*:*:instance/*"
11         },
12         {
13             "Effect": "Allow",
14             "Action": "ec2:DescribeInstances",
15             "Resource": "*"
16         }
17     ]
18 }

```

3. Lambda Function:

- Navigate to the Lambda dashboard and create a new function.
- Choose Python 3.x as the runtime.
- Assign the IAM role created in the previous step.
- Write the Boto3 Python script to:
 1. Initialize a boto3 EC2 client.
 2. Describe instances with `Auto-Stop` and `Auto-Start` tags.
 3. Stop the `Auto-Stop` instances and start the `Auto-Start` instances.
 4. Print instance IDs that were affected for logging purposes.



4. Manual Invocation:

- After saving your function, manually trigger it.
- Go to the EC2 dashboard and confirm that the instances' states have changed according to their tags.

```

testing@inblrlt-002: ~$ aws ec2 describe-instances --query "Reservations[*].Instances[*].(PublicAddress:PublicIpAddress,PrivateAddress:PrivateIpAddress,InstanceId:InstanceId,Platform:Platform,State,Name,Name:Tags[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value)" --output table
DescribeInstances
+-----+-----+-----+-----+-----+-----+-----+
| Action | InstanceID | Name | Platform | PrivateAddress | PublicAddress | State |
+-----+-----+-----+-----+-----+-----+-----+
| None | i-037f542b93fe8accd | frontend-01 | Linux/UNIX | 172.31.12.201 | None | stopped |
| None | i-0400124bba6643b1a | frontend-02 | Linux/UNIX | 172.31.13.171 | None | stopped |
| Auto-Stop | i-011facb3f85dcfcde | dev-server | Linux/UNIX | 172.31.8.179 | 65.0.173.7 | running |
| Auto-Start | i-03cea284f141c9cba | prod-server | Linux/UNIX | 172.31.15.137 | None | stopped |
+-----+-----+-----+-----+-----+-----+-----+

testing@inblrlt-002: ~$ aws lambda invoke \
--function-name arn:aws:lambda:ap-south-1:135808938060:function:stop-start-ec2 \
--cli-binary-format raw-in-base64-out \
--payload '{ "key": "value" }' \
response.json
{"statusCode": 200,
"ExecutedVersion": "$LATEST"}

testing@inblrlt-002: ~$ aws ec2 describe-instances --query "Reservations[*].Instances[*].(PublicAddress:PublicIpAddress,PrivateAddress:PrivateIpAddress,InstanceId:InstanceId,Platform:Platform,State,Name,Name:Tags[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value)" --output table
DescribeInstances
+-----+-----+-----+-----+-----+-----+-----+
| Action | InstanceID | Name | Platform | PrivateAddress | PublicAddress | State |
+-----+-----+-----+-----+-----+-----+-----+
| None | i-037f542b93fe8accd | frontend-01 | Linux/UNIX | 172.31.12.201 | None | stopped |
| None | i-0400124bba6643b1a | frontend-02 | Linux/UNIX | 172.31.13.171 | None | stopped |
| Auto-Stop | i-011facb3f85dcfcde | dev-server | Linux/UNIX | 172.31.8.179 | 65.0.173.7 | stopping |
| Auto-Start | i-03cea284f141c9cba | prod-server | Linux/UNIX | 172.31.15.137 | 15.207.71.115 | running |
+-----+-----+-----+-----+-----+-----+-----+

testing@inblrlt-002: ~$ aws ec2 describe-instances --query "Reservations[*].Instances[*].(PublicAddress:PublicIpAddress,PrivateAddress:PrivateIpAddress,InstanceId:InstanceId,Platform:Platform,State,Name,Name:Tags[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value)" --output table
DescribeInstances
+-----+-----+-----+-----+-----+-----+-----+
| Action | InstanceID | Name | Platform | PrivateAddress | PublicAddress | State |
+-----+-----+-----+-----+-----+-----+-----+
| None | i-037f542b93fe8accd | frontend-01 | Linux/UNIX | 172.31.12.201 | None | stopped |
| None | i-0400124bba6643b1a | frontend-02 | Linux/UNIX | 172.31.13.171 | None | stopped |
| Auto-Stop | i-011facb3f85dcfcde | dev-server | Linux/UNIX | 172.31.8.179 | None | stopped |
| Auto-Start | i-03cea284f141c9cba | prod-server | Linux/UNIX | 172.31.15.137 | 15.207.71.115 | running |
+-----+-----+-----+-----+-----+-----+-----+

testing@inblrlt-002: ~$ ls
Desktop Documents Downloads fbs index.html kubect1 kubect1.sha256 Music Pictures Public response.json skil snap study Templates Videos
testing@inblrlt-002: ~$ cat response.json
{"statusCode": 200, "body": "{\"StoppedInstances\": [\"i-011facb3f85dcfcde\"], \"StartedInstances\": [\"i-03cea284f141c9cba\"]}"}
testing@inblrlt-002: ~$

```

Assignment 2: Automated S3 Bucket Cleanup Using AWS Lambda and Boto3

Objective: To gain experience with AWS Lambda and Boto3 by creating a Lambda function that will automatically clean up old files in an S3 bucket.

Task: Automate the deletion of files older than 30 days in a specific S3 bucket.

Instructions:

1. S3 Setup:

- Navigate to the S3 dashboard and create a new bucket.
- Upload multiple files to this bucket, ensuring that some files are older than 30 days (you may need to adjust your system's date temporarily for this or use old files).

2. Lambda IAM Role:

- In the IAM dashboard, create a new role for Lambda.
- Attach the `AmazonS3FullAccess` policy to this role. (Note: For enhanced security in real-world scenarios, use more restrictive permissions.)

3. Lambda Function:

- Navigate to the Lambda dashboard and create a new function.
- Choose Python 3.x as the runtime.
- Assign the IAM role created in the previous step.
- Write the Boto3 Python script to:
 1. Initialize a boto3 S3 client.
 2. List objects in the specified bucket.
 3. Delete objects older than 30 days.
 4. Print the names of deleted objects for logging purposes.

4. Manual Invocation:

- After saving your function, manually trigger it.

- Go to the S3 dashboard and confirm that only files newer than 30 days remain.

Assignment 3: Monitor Unencrypted S3 Buckets Using AWS Lambda and Boto3

Objective: To enhance your AWS security posture by setting up a Lambda function that detects any S3 bucket without server-side encryption.

Task: Automate the detection of S3 buckets that don't have server-side encryption enabled.

Instructions:

1. S3 Setup:

- Navigate to the S3 dashboard and create a few buckets. Ensure that a couple of them don't have server-side encryption enabled.

2. Lambda IAM Role:

- In the IAM dashboard, create a new role for Lambda.
- Attach the `AmazonS3ReadOnlyAccess` policy to this role.

3. Lambda Function:

- Navigate to the Lambda dashboard and create a new function.
- Choose Python 3.x as the runtime.
- Assign the IAM role created in the previous step.
- Write the Boto3 Python script to:
 1. Initialize a boto3 S3 client.
 2. List all S3 buckets.
 3. Detect buckets without server-side encryption.
 4. Print the names of unencrypted buckets for logging purposes.

4. Manual Invocation:

- After saving your function, manually trigger it.

- Review the Lambda logs to identify the buckets without server-side encryption.

Assignment 6: Monitor and Alert High AWS Billing Using AWS Lambda, Boto3, and SNS

Objective: Create an automated alerting mechanism for when your AWS billing exceeds a certain threshold.

Task: Set up a Lambda function to check your AWS billing amount daily, and if it exceeds a specified threshold, send an alert via SNS.

Instructions:

1. SNS Setup:

- Navigate to the SNS dashboard and create a new topic.
- Subscribe your email to this topic.

2. Lambda IAM Role:

- In the IAM dashboard, create a new role for Lambda.
- Attach policies that allow reading CloudWatch metrics and sending SNS notifications.

3. Lambda Function:

- Navigate to the Lambda dashboard and create a new function.
- Choose Python 3.x as the runtime.
- Assign the IAM role created in the previous step.
- Write the Boto3 Python script to:
 1. Initialize boto3 clients for CloudWatch and SNS.
 2. Retrieve the AWS billing metric from CloudWatch.
 3. Compare the billing amount with a threshold (e.g., \$50).
 4. If the billing exceeds the threshold, send an SNS notification.

5. Print messages for logging purposes.

4. Event Source (Bonus):

- Attach an event source, like Amazon CloudWatch Events, to trigger the Lambda function daily.

5. Testing:

- Manually trigger the Lambda function or wait for the scheduled event.
- If your billing is over the threshold, you should receive an email alert.

Assignment 12: Auto-Scale EC2 Instances Based on Load Using AWS Lambda, Boto3, and SNS

Objective: Automatically scale up or down the number of EC2 instances based on network load.

Task: Set up a Lambda function that checks the network load on an ELB and scales the number of EC2 instances accordingly.

Instructions:

1. SNS Setup:

- Create a new SNS topic.
- Subscribe to the topic.

2. Lambda IAM Role:

- Create a role with permissions to read ELB metrics, start/terminate EC2 instances, and send SNS notifications.

3. Lambda Function:

- Create a function with the above IAM role.
- Use Boto3 to:
 1. Fetch the network load metric from ELB.
 2. If the load is above a high threshold (e.g., 80% for 5 minutes), launch a new EC2 instance.

3. If the load is below a low threshold (e.g., 20% for 5 minutes), terminate an EC2 instance.

4. Notify via SNS about the scaling action taken.

4. CloudWatch Events:

- Schedule your Lambda function to run every 5 minutes.

5. Testing:

- Simulate varying network loads.
- Confirm that EC2 instances are correctly scaled and SNS notifications are received.

Assignment 14: Monitor EC2 Instance State Changes Using AWS Lambda, Boto3, and SNS

Objective: Automatically monitor changes in EC2 instance states and send notifications whenever an instance is started or stopped.

Task: Set up a Lambda function that listens to EC2 state change events and sends SNS notifications detailing the state changes.

Instructions:

1. SNS Setup:

- Navigate to the SNS dashboard and create a new topic.
- Subscribe to this topic with your email.

2. Lambda IAM Role:

- Create a role with permissions to read EC2 instance states and send SNS notifications.

3. Lambda Function:

- Create a function and assign the above IAM role.
- Use Boto3 to:

1. Extract details from the event regarding the EC2 state change.

2. Send an SNS notification with details about which EC2 instance changed state and the new state (e.g., started, stopped).

4. EC2 Event Bridge (formerly CloudWatch Events):

- Set up an Event Bridge rule to trigger your Lambda function whenever an EC2 instance state changes.

5. Testing:

- Start or stop one of your EC2 instances.
- Confirm you receive an SNS notification about the state change.

Assignment 18: Autosave EC2 Instance State Before Shutdown

Objective: Before an EC2 instance is shut down, automatically save its current state to an S3 bucket.

Instructions:

1. Create a Lambda function.

2. Using Boto3, the function should:

1. Detect when an EC2 instance is about to be terminated.
2. Save the current state or necessary files from the EC2 instance to an S3 bucket.

3. Use CloudWatch Events to trigger this Lambda function whenever an EC2 termination command is detected.

Assignment 19: Load Balancer Health Checker

Objective: Design a Lambda function that checks the health of registered instances behind an Elastic Load Balancer (ELB) and notifies via SNS if any instances are unhealthy.

Instructions:

1. Create a Lambda function.

2. With Boto3, configure the function to:

1. Check the health of registered instances behind a given ELB.

2. If any instances are found to be unhealthy, publish a detailed message to an SNS topic.
3. Set up a CloudWatch event to trigger this Lambda function every 10 minutes.